FACULTAD DE INGENIERÍA Y CIENCIAS

ESCUELA DE INFORMÁTICA Y TELECOMUNICACIONES

Informe Tarea 2: Torneo de Komikan Concurrente

Sistemas Operativos

Sección: 04

Profesor: Victor Reyes

Alumno: Nahuel Sánchez

Índice

1 Diseño del esquema de sincronización **1**

2 Análisis de riesgos de concurrencia **2**

3 Evaluación **2**

1 Diseño del esquema de sincronización

Arquitectura. El programa (`torneo.c`) es un único proceso con hilos POSIX, y el tablero de cada mesa es memoria compartida por los hilos de esa partida.

- **main:** valida la entrada, crea los demás hilos y espera señales con `sigwait`.
- **N hilos jugador:** cada uno llega tras un retardo aleatorio, entra a una cola FIFO y espera su resultado.
- **Matchmaker:** saca dos jugadores de la cola (el primero juega como Puma y el segundo como Perros) y les asigna una mesa libre.
- **M jueces:** uno por mesa. Cada uno inicializa el tablero y sus primitivas, crea las 13 piezas, espera el fin de la partida, hace `join` de las piezas, destruye las primitivas y libera la mesa.
- **13 hilos-pieza por partida** (Puma + 12 Perros), que juegan sin turnos.

El tablero es una lista de adyacencias de 28 nodos: la grilla de 5×5, con conexiones solo horizontales y verticales, más los 3 nodos del Trono, conectados según la figura del enunciado. Los saltos de captura (A→B→C en línea recta) se precalculan al inicio.

Primitivas POSIX y qué protegen.

`srv_mtx` (mutex)	Protege la cola, `mesas_libres` y el estado de jugadores y mesas. Una mesa solo se asigna si `mesas_libres > 0`, y el contador se decrementa en la misma sección crítica; por eso nunca hay más de M mesas activas.
`cv_mm`, `jugador.cv`, `mesa.cv_asignada`	**Espera pasiva**. El matchmaker duerme mientras no haya "≥ 2 en cola y una mesa libre"; cada jugador duerme hasta recibir su resultado, y cada juez hasta que le asignan una pareja. Todas usan `while(!condición) pthread_cond_wait(...)`.
`mesa.mtx` (uno por mesa)	Protege el tablero de esa partida. Es un mutex **robusto** (`PTHREAD_MUTEX_ROBUST`).
`mesa.cv`	Las piezas duermen su retardo con `pthread_cond_timedwait`. Al terminar la partida se hace `broadcast` y todas despiertan.
`log_mtx`	Evita que se mezclen las líneas impresas por distintos hilos.

Exclusión mutua en movimientos simultáneos. En cada ciclo, una pieza hace lo siguiente:

1. Espera un retardo aleatorio entre `DELAY_MIN` y `DELAY_MAX`.
2. Copia el tablero bajo el mutex y lo suelta.
3. Decide su jugada sobre la copia, fuera de la sección crítica.
4. Vuelve a tomar el mutex y llama a `mover`.

`mover` valida contra el estado *actual*: que la pieza siga viva y en su origen, y que el destino sea adyacente y esté libre (o que sea un salto del Puma sobre un perro). Luego aplica el movimiento y revisa la victoria: el Puma gana con 6 capturas y los Perros ganan si el Puma queda sin movimientos. Todo esto ocurre en la misma sección crítica. Si dos piezas apuntan a la misma casilla, el mutex las atiende de a una: la primera la obtiene y la segunda recibe `COLISION`. Por eso nunca hay dos piezas en una casilla ni capturas dobles.

Fallos y SIGINT.

- **Fallo de un hilo-pieza:** si un hilo muere con el mutex tomado, el siguiente `lock` devuelve `EOWNERDEAD`. Se llama a `pthread_mutex_consistent` para recuperar el mutex y el rival gana por abandono. Al final de la partida, el juez destruye el mutex.
- **SIGINT:** `SIGINT` y `SIGUSR1` se bloquean con `pthread_sigmask` antes de crear los hilos, y `main` las espera con `sigwait`. Ante `SIGINT`, `main` marca `cierre`, despierta todas las condiciones y termina las partidas en curso. Luego hace `join` de todos los hilos, destruye las primitivas y libera la memoria.
- **Fin normal:** el último jugador en terminar envía `SIGUSR1`, que despierta a `main`.

2 Análisis de riesgos de concurrencia

Riesgo	Escenario	Prevención
Deadlock	Ante `SIGINT`, `main` toma `srv_mtx` y después el mutex de cada mesa, mientras otro hilo podría tomarlos en orden inverso.	Orden fijo `srv_mtx`→`mesa.mtx`→`log_mtx`: ningún hilo toma `srv_mtx` mientras tiene el mutex de una mesa.
Deadlock por hilo muerto	Una pieza muere con el mutex del tablero tomado y deja bloqueadas a las demás y al juez.	Mutex robusto + `EOWNERDEAD` + `pthread_mutex_consistent`.
Livelock	Dos piezas apuntan a la misma casilla y reintentan una y otra vez al mismo tiempo.	Cada pieza reintenta con su propio retardo aleatorio, así que tarde o temprano una gana.
Starvation	Una pieza que nunca consigue el mutex, o jugadores que nunca reciben mesa.	El mutex se retiene microsegundos (la decisión se toma fuera), frente a retardos de milisegundos. La cola es FIFO.
Bloqueo eterno	N impar, o `SIGINT` con jugadores en la cola.	El último jugador de un N impar recibe "sin rival", la cola se vacía con "servidor cerrado" y toda espera revisa `cierre`.
Uso tras destruir	Llega `SIGINT` mientras un juez destruye el mutex de su mesa.	El juez pone `en_juego = false` bajo `srv_mtx` antes de destruirlo, y el cierre solo toca mesas con `en_juego`.

3 Evaluación

Complejidad vs. concurrencia. Se usa **un mutex por mesa**. Un mutex por casilla no conviene: una captura toca 3 casillas y la victoria depende de todo el tablero, así que habría que tomar varios locks a la vez (más riesgo de deadlock) para una ganancia mínima. Un único mutex global tampoco conviene, porque detendría mesas que no tienen relación entre sí. Con un mutex por mesa, las M partidas avanzan en paralelo, y dentro de cada mesa la sección crítica es muy corta, porque las piezas deciden fuera de ella. Se eligió mutex + variable de condición en lugar de semáforos porque las esperas dependen de condiciones compuestas ("2 en cola y una mesa libre, o cierre") y deben poder despertarse todas ante `SIGINT`.

Rendimiento. La latencia de un movimiento la determina el retardo configurado (50–300 ms); la espera por el mutex es despreciable. Con 20 jugadores y 2 mesas, los jugadores en cola esperaron varios segundos y el proceso usó $\approx$1 % de CPU, lo que confirma que la espera es pasiva. Con retardos de 1–3 ms aparecen colisiones reales, y en cada una se acepta un solo movimiento.

Verificación. Se realizaron 26 verificaciones y todas pasaron: partida normal, saturación, colisiones, fallos de un hilo, N impar, Ctrl+C real, 12 entradas inválidas y valgrind sin fugas, con y sin `SIGINT`. AddressSanitizer y ThreadSanitizer tampoco reportaron errores.